LAB – ELECTION ALGORITHMS

Student Details:

Name: Louis Muiyuro

Admission Number: 169275

Unit: Distributed Systems (ICS 4104)

Title: Leader Election Using the Bully and Ring Algorithms

Group Number: 14

Theory:

In a distributed system, several independent processes sometimes need one of them to act as a coordinator, the process responsible for tasks such as sequencing transactions or managing shared resources. As long as the coordinator is alive, no election is needed. The problem arises when the coordinator crashes or becomes unreachable: the remaining processes must agree on a replacement without any central authority telling them who it is. Election algorithms are the protocols used to reach this agreement.

Both algorithms covered in this lab share the same underlying assumption: every process has a unique, comparable ID, and the process with the highest ID among the survivors always becomes the new coordinator. They differ only in how the processes discover and agree on this.

The Bully Algorithm

When a process notices that the coordinator is not responding, it starts an election by contacting only the processes with a higher ID than itself. If none of them respond, it declares itself the coordinator. If a higher process responds, that process takes over the election. This repeats until the highest-ID process still alive becomes coordinator ,it “bullies” its way past every lower-ranked process.

The Ring Algorithm

Processes are arranged in a logical ring, where each process only knows its next neighbour. An election message carrying an ID is passed around the ring. Each process that receives it compares the ID it carries to its own and keeps the larger one before forwarding it on. When the message returns to the process that started the election, it carries the maximum ID in the ring, which is then announced as the new coordinator by circulating a coordinator message around the ring.

Aim:

To implement and demonstrate the Bully and Ring algorithms for leader election in a distributed system, and to observe how each algorithm arrives at the same outcome, the highest-ID surviving process becoming coordinator, through different message-passing strategies.

Algorithm:

(a) Bully Algorithm

- Assign each process a unique ID, with one process initially acting as coordinator.
- When a process notices the coordinator is unreachable, it starts an election.
- The process sends an ELECTION message to every process with a higher ID than itself.
- If no higher-ID process responds, the process declares itself coordinator and broadcasts this to all processes.
- If a higher-ID process responds, that process takes over the election in its place.
- This continues until the highest-ID process still active becomes the coordinator.
- If a higher-ID process later comes back online, it re-triggers an election and reclaims coordinator status.

(b) Ring Algorithm

- Arrange all participating processes into a logical ring, each aware only of its next neighbour.
- A process that notices the coordinator has failed starts an election by sending an ELECTION message, containing its own ID, to its neighbour.
- Each process that receives the message compares the ID carried in the message with its own ID and forwards the larger of the two to its neighbour.
- The message continues travelling around the ring, picking up the highest ID seen so far.
- When the message returns to the process that started the election, it holds the maximum ID in the ring.
- The initiating process announces this ID as the new coordinator by circulating a COORDINATOR message around the ring.

Implementation:

The lab was implemented using Java in a Windows PowerShell environment. Unlike a networked implementation using sockets, both algorithms here are simulated within a single Java program

each — process state (active/inactive) is held in memory and the message exchange between processes is represented by console output. Two separate programs were used:

- Bully.java — simulates 5 fixed processes and the Bully election protocol.
- Ring.java (with helper class Rr) — simulates a configurable number of processes arranged in a ring, running the Ring election protocol.

(a) Software Requirements:

- Java Development Kit (JDK) 17 or compatible (JDK 23 used)
- Java Compiler (javac)
- Java Runtime (java)
- Terminal / PowerShell command line interface

(b) Compilation:

Both source files were compiled from the lab folder using:

```
C:\Users\Muiyuro>cd "C:\Users\Muiyuro\Documents\Strathmore_University\Ye
ar 4.1\Distributed systems\labs\Election Algorithm Bully and Ring algori
thm for leader election"

C:\Users\Muiyuro\Documents\Strathmore_University\Year 4.1\Distributed sy
stems\labs\Election Algorithm Bully and Ring algorithm for leader electi
on>javac *.java

C:\Users\Muiyuro\Documents\Strathmore_University\Year 4.1\Distributed sy
stems\labs\Election Algorithm Bully and Ring algorithm for leader electi
on>dir *.class
 Volume in drive C has no label.
 Volume Serial Number is 1A67-D185

 Directory of C:\Users\Muiyuro\Documents\Strathmore_University\Year 4.1\
Distributed systems\labs\Election Algorithm Bully and Ring algorithm for
 leader election

19/07/2026  01:21             2,919 Bully.class
19/07/2026  01:21             2,701 Ring.class
19/07/2026  01:21               262 Rr.class
               3 File(s)          5,882 bytes
               0 Dir(s)   4,624,363,520 bytes free

C:\Users\Muiyuro\Documents\Strathmore_University\Year 4.1\Distributed sy
stems\labs\Election Algorithm Bully and Ring algorithm for leader electi
on>
```

This produced Bully.class, Ring.class, and Rr.class alongside the .java source files.

(c) Execution — Bully Algorithm:

The program was run and the following scenario was demonstrated: process 5 (the coordinator) is brought down, then process 3 attempts to send a message. Since the coordinator is unreachable, this triggers a full election.

```
C:\Users\Muiyuro\Documents\Strathmore_University\Year 4.1\Distributed sy
stems\labs\Election Algorithm Bully and Ring algorithm for leader electi
on>java Bully
5 active process are:
Process up  = p1 p2 p3 p4 p5
Process 5 is coordinator
.........
1) Up a process.
2) Down a process
3) Send a message
4) Exit
2
Bring down any process.
5
.........
1) Up a process.
2) Down a process
3) Send a message
4) Exit
3
Which process will send message
3
Process 3 election
Election send from process 3 to process 4
Election send from process 3 to process 5
Coordinator message send from process 4 to all
.........
1) Up a process.
2) Down a process
3) Send a message
4) Exit
4
```

As expected under the Bully rule, process 4 — the highest surviving ID once process 5 went down — is declared the new coordinator.

(d) Execution — Ring Algorithm:

The program was run with 5 processes (IDs 1 to 5) arranged in a ring. Process 5 is automatically assigned as the initial coordinator. An election was then triggered from the process at index 3, and the message was passed around the ring until it returned with the highest surviving ID.

```
C:\Users\Muiyuro\Documents\Strathmore_University\Year 4.1\Distributed sy
stems\labs\Election Algorithm Bully and Ring algorithm for leader electi
on>java Ring
Enter the number of process :
5
Enter the id of process :
1
Enter the id of process :
2
Enter the id of process :
3
Enter the id of process :
4
Enter the id of process :
5
  [0] 1  [1] 2  [2] 3  [3] 4  [4] 5
 process 5 select as co-ordinator

 1.election 2.quit
1

 Enter the Process number who initialsied election :
3

Process 4 send message to 1

Process 1 send message to 2

Process 2 send message to 3

Process 3 send message to 4

 process 4 select as co-ordinator

 1.election 2.quit
2
Program terminated ...
```

The election message travelled around the ring collecting IDs, and process 4 — the highest ID among the still-eligible processes (process 5 having already served as coordinator) — was correctly elected as the new coordinator.

Comparison of the Two Algorithms:

Aspect	Bully	Ring
Topology assumption	Every process knows every other process	Each process only knows its next neighbour
Message direction	Sent to all higher-ID processes	Passed sequentially around the ring
Message count	Higher — up to $O(n^2)$ worst case	Lower — up to O(2n) per election

Resolution speed	Can resolve quickly	Slower — must traverse the full ring
Outcome	Highest surviving ID becomes coordinator	Highest ID collected while circling becomes coordinator

Conclusion:

This lab demonstrated how distributed systems can recover from coordinator failure without centralised control, using two different message-passing strategies. The Bully algorithm resolves elections quickly by directly contacting higher-ranked processes but requires full network knowledge, while the Ring algorithm requires only neighbour-level knowledge at the cost of a full traversal per election. Running both simulations confirmed that each algorithm correctly identifies the highest-ID surviving process as the new coordinator, verifying that both approaches achieve the same distributed consensus outcome through structurally different means.